INTRODUCTION

CacheWise replication and KV cache recovery

Replicating CacheWise and evaluating host-DRAM recovery

CENTRAL QUESTION

How can coding agents reuse previous computation when GPU memory is limited?

What do the CacheWise traces contain?

CATraces: real coding sessions

Claude Code interaction logs
Model requests + tool calls
Timing + prompt / output lengths

Released content is redacted; replay preserves lengths and prefix growth.

Reuse across turns

Growing, shared prompt prefixes
Long inputs with short outputs
Tool waits between requests

Requests arrive in sessions, with repeated opportunities for reuse.

Why prefix-aware scheduling can help

Reuse a resident prefix → less new prefill → fewer additional KV blocks.
A smaller active working set can reduce eviction pressure.

CacheWise v1 §§3–5; released-trace replay: report §3.1

CacheWise: scheduling and eviction

Prefix-aware scheduling

Which request goes next?

Favor requests needing fewer additional KV blocks.

Uses GPU prefix overlap.

Predictive eviction

Which cached state should stay?

Use tool metadata to estimate when inactive sessions return.

Uses predicted time until reuse.

Scheduling chooses what to serve; eviction chooses what to reclaim.

Our replay tests these policies before the separate host-recovery study.

Replication separates two decisions

Scheduler × eviction policy

	LRU	Predictive
FCFS	**FCFS / LRU**	**FCFS / predictive**
Prefix-aware	**Prefix / LRU**	**Prefix / predictive**

Coverage study

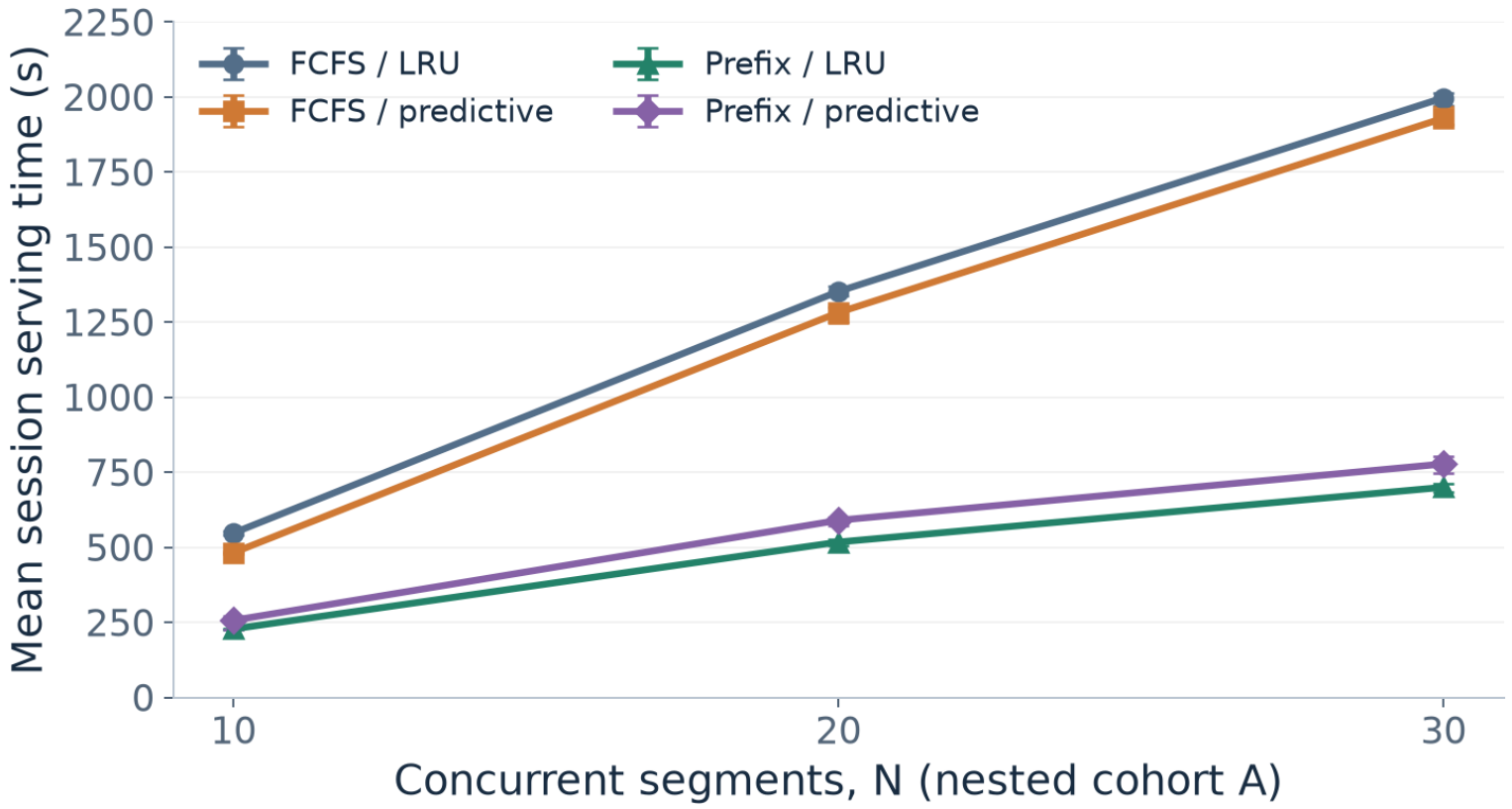
5 settings × 3 repeats
60 runs · 6,384 requests

14B model · BF16
287,696 KV token slots
512-token prefill chunks

Primary metric: mean session serving time, excluding tool waits.

Evidence: part1-coverage-comparison.json; report §3.1–3.2

Prefix / LRU has the lowest mean



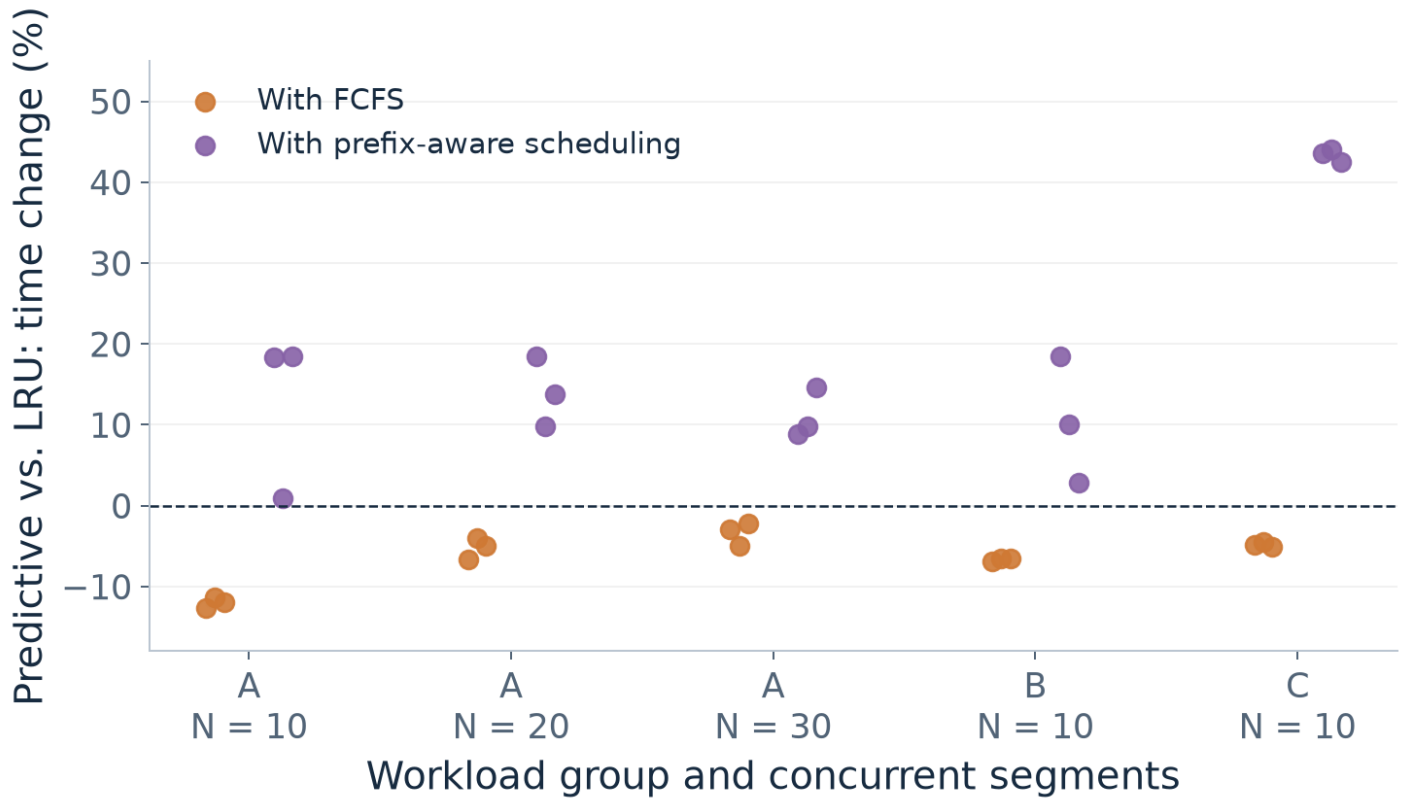
5 / 5

settings favor prefix / LRU

58–65% lower mean vs. FCFS / LRU in cohort A

Three repeats per cell. Ranges are observed minima/maxima; lower is better.

Prediction depends on the scheduler



Workload key

A: load sweep

10 → 20 → 30 segments
Keep earlier segments.

B/C: extra checks

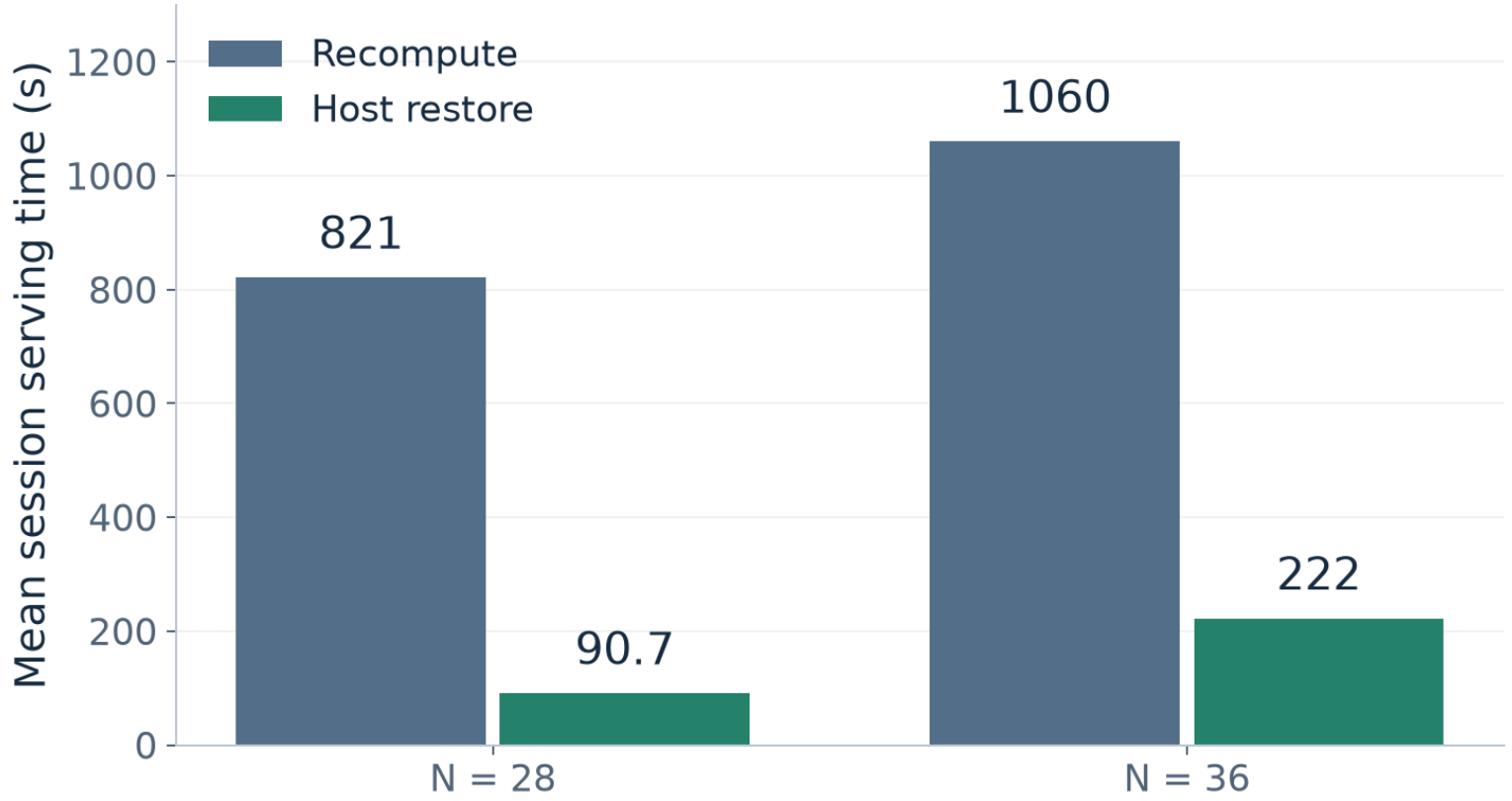
10 segments each;
different source sessions
from A and one another.

N = concurrent segments

Dots are matched-repeat effects. Positive values mean slower serving.

Host restoration reduces session time

Recovery tradeoff: save + restore + management < recomputation



9.1× / 4.8×

reported time ratios
at N = 28 / 36

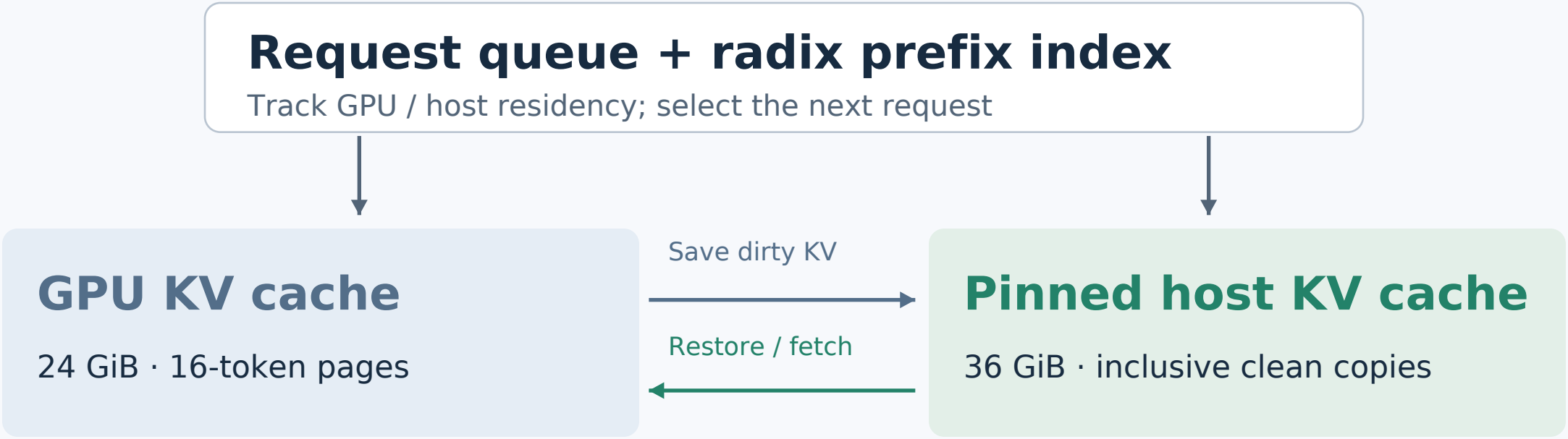
+36 GiB host KV
24 GiB GPU KV
fixed

One observation
per bar

Separate Mini-SGLang study. Added-capacity comparison; lower is better.

Evidence: host-dram-reported-results.json; report Figure 3a and §6

Preserve evicted KV in host DRAM



On demand

Restore before prefill.

Prefetch

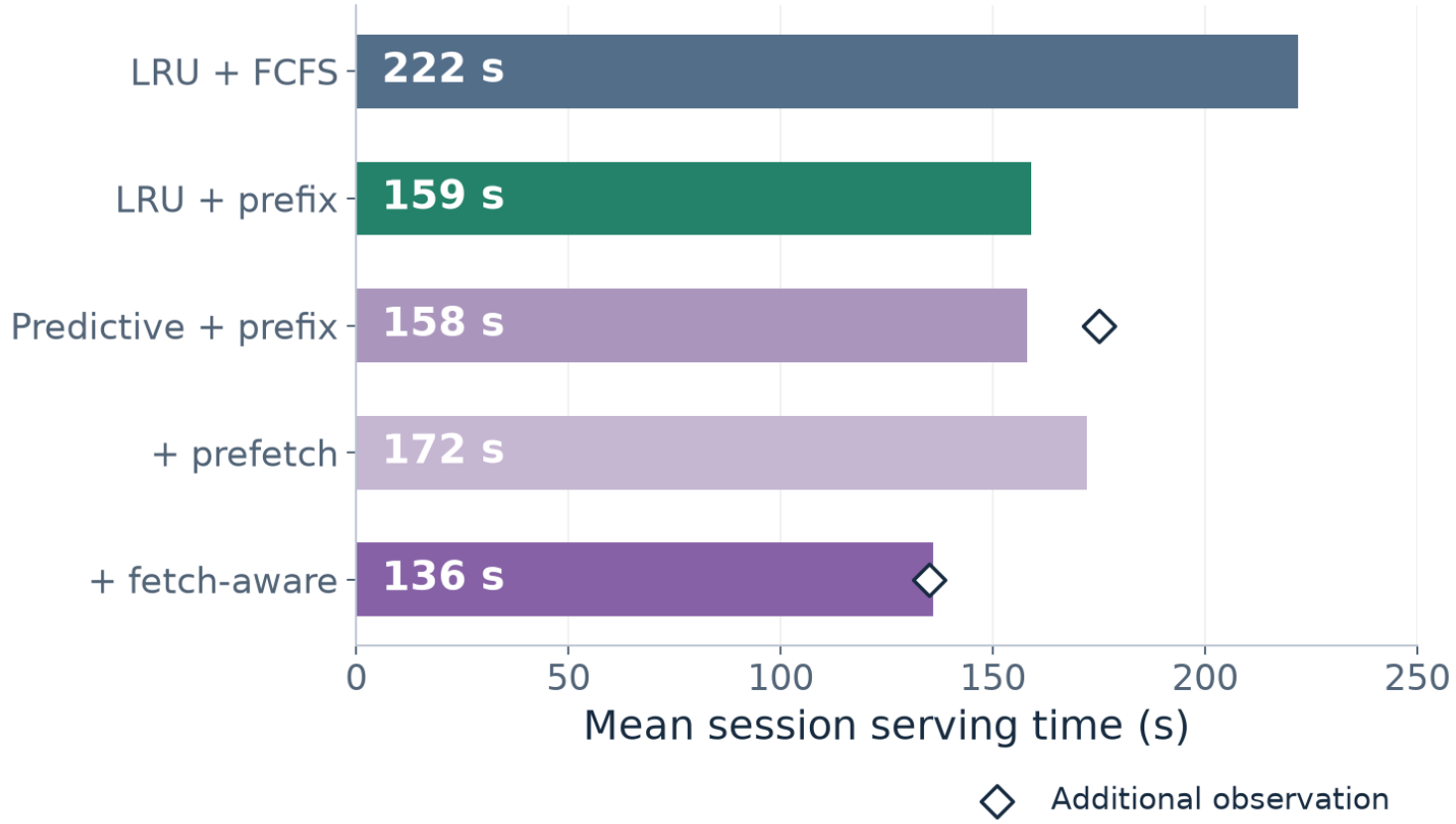
Overlap predicted reuse.

Fallback

Recompute on a miss.

Implementation: Mini-SGLangs 3f9aee7; report Figure 2 and §5

Fetch-aware ordering discounts host hits



Priority cost

new prefill tokens + 0.05 × host tokens

Mean of 2 observations
166.5 s → 135.5 s

Sparse repetitions; no confidence interval.

Bars: first observations. Diamonds: additional observations, not matched blocks.

Evidence: Mini-SGLangs 3f9aee7; report Figure 3b and Eq. (3)

Keep the conclusions within the evidence

Replication

Synthetic replay and selected segments.

One retained preemption; extra prefill work unmeasured.

Host results

Separate engine and replay.

Added host capacity; most configurations run once.

SSD extension

Not implemented or measured.

Needs full I/O, staging, metadata and recovery.

Interpret both studies separately; retain the declared protocol deviation.

Three takeaways

01 Scheduling and eviction interact.

Prefix / LRU has the lowest mean in all five replication settings.

02 Host copies can avoid costly recovery.

Large reported gains include the added 36 GiB backing tier.

03 Order requests by recovery cost.

Fetch-aware results are promising; SSD evaluation remains future work.